Memory of some technical updating process

Nareli Cruz Cortés

July 19, 2026

Contents

1 Some Code in Python

First I am using Visual Studio Code. I open the folder (Very important to open the folder, not the file!), then create a virtual environment to avoid conflicts with global variables, and version and packages. In the terminal open the same folder and use the command[1]:

```
# Only to show python is alive
python3 --version
# This creates a virtual environment in a folder named venv
python3 -m venv venv
# After the Virtuall Environment was created, it must be activated
source venv\bin\activate
```

All the project should be placed in the same folder.
Then, all the packages can be installed under this venv in the same working folder. To install packages use the terminal. Take these as examples:
pip install matplotlib
pip install scikit-learn
...
Pip means something like python installer packages... i dont know...

To verify what the libraries are actually installed use the terminal. Note: All files must be in the same directory which is the one with the active venv:
To display the installed libraries: `pip list`
To verify the python is active: `which python`
or `which python3`
The pip can be checked as well: `which pip`
There are several python interpreters, some global or in other virtual spaces, if it is the case that the current one is not in use, it can be selected with:
`command (Flower)+Shift+P` then select it

If all the libraries are there, they can be saved in a file, so next time there will not be need to install them one by one:
pip freeze> requirements.txt

[1]The -m comes from module.

When finished the whole thing, deactivate the virtual environment using in the terminal: `deactivate`. Most times it is not necessary.

In scikit-learn, the typical code is

- from sklearn.linear_model import LinearRegression
- Load data for training, for example numpy arrays for vectors X and y.
- Create the model, that means to instantiate the object, for example: model = LinearRegression()
- model.fit(X,y)
- model.predict

1.1 Jupyter Notebook

This is a format that allows interactively running *cells* which are pieces of code, of python in this case. The cells are run one by one. To be sure that the active venv is the correct one, use this code inside a cell:

```
import sys
print(sys.executable)
```

the output of that cell must coincide with the venv directory. If some errors on this topic, probably the next library is missing:
`pip install ipykernel`

I can write and run the notebook using Visual Studio Code, or alternatively I can run it by writing in a terminal: `jupyter notebook`
This will open the jupyter notebook in a web browser. It will be listening to port 8888

If the Notebook in Visual Studio Code is running a different kernel, the solution is to include it in the current venv, with the next instruction:
`python -m ipykernel install --user --name=venv`
Explanation:

- Run ipykernel module as a program
- If the venv is active, then python refers to: project/venv/bin/python and that is the python that is registered.

- –user install the kernel only for my user account
- –name=venv gives the kernel an internal name venv

For a tradicional python file script the name extension should be .py
For the case of a Jupyter Notebook the extension name shoudl be .ipynb

2 To Publish Code

The code must be upload to be available to everyone, that is the natural thing to do. It looks necessary for any kind of modern approach.

2.1 GIT

The account in GitHub is using my e-mail in the domain protonmail. When everything runs according to the previous sections, it is necessary to convert the regular folder where all the files are set, into a GIT repository. That means it will be able to keep versions and tracks changes and collaborators, branches, tags.
It means that instead of keeping my versions in files with names as:
exerciseFinal1.ipynb, ...exerciseFinal2.ipynb ... exercise_Final3.ipynb ... and so on, GIT will maintain all the versions without the mess of the names.
To do it, run in a terminal the next command:
`git init`
Then, in the terminal I changed the default name, which was master, to a nicer more ecological one:

```
 git branch -m trunk
git add .
git commit -m "Initial commit"
```

All these can be used locally. It is necessary to apply *commit* every certain time after important changes were applied to the project. The local folder looks like this:

```
 drwxr-xr-x   8 elicres  staff    256 Jun  5 19:34 .
drwxr-xr-x@ 19 elicres  staff    608 Jun  5 11:30 ..
drwxr-xr-x  12 elicres  staff    384 Jun  5 19:55 .git
-rw-r--r--   1 elicres  staff     55 Jun  5 11:08 .gitignore
-rw-r--r--   1 elicres  staff     48 Jun  5 14:33 README.md
-rw-r--r--@  1 elicres  staff  27139 Jun  5 14:56 linearReg.ipynb
```

```
-rw-r--r--   1 elicres  staff    777 Jun  5 14:54 requirements.txt
drwxr-xr-x@  7 elicres  staff    224 Jun  4 21:06 venv
```

2.2 Publish it

The code will be published in GitHub. It can be run there BUT it is not the point! The point is to keep the versions and allow that people download or cloned it. So they can run or modify it in their own computers.

To publish the code I used GitHub.com . The email's account is elicres@protonmail.com
`https://github.com/Elinada/my-first-notebooks.git`
For a Jupyter Notebook, the general steps are as follows:

1. All the code is prepared locally in my laptop. Specifically the next files:

   - mycode.ipynb
   - README.dm

     ```
     # installation
     pip install -r requirements.txt
     ```

   - **requirements.txt**
   - **.gitignore**
     In a terminal create an empty file with:
     `touch .gitignore`
     `nano .gitignore`
     then inside the file write the following:

     ```
     .ipynb_checkpoints/
     __pycache__/
      *.pyc
     .DS_Store
     venv/
     ```

     CTRL+O to overwrite then close it.

   Inside the GitHub.com create a **New Repository**, give it a name and create it.
   For the first time write these in the local terminal:

```
git init
git add .
git commit -m "Initial commit"
```

For the remote access, these are the new commands:

```
git remote add origin <url>
git branch -M main
git push -u origin main
```

"git branch -M main" This is needed only to rename the branch name to *main*. To update in next events in the GitHub web, type this locally:

```
git add .
git commit -m "change"
git push
```

If necessary to verify the status, do it in the terminal with: **git status**
git remote -v

Every time I need to update only do this:
git add .
git commit -m "change"

HACE FALTA DECIR COMO SE HACE PARA RECUPERAR UNA VERSION ANTERIOR

2.3 Download or Clone

For the case that I am downloading some code the steps are the following:

- In the github.com there is a green bouton with CODE, there is a URL to be copied
- Then clone the code from my local terminal:
  `git clone https://github.com/user/project.git`

Everything is cloned in my laptop. Now the venv should be activated.

2.4 Throubleshouting

I had some problems installing this, some possibles causes are the folllowing:

1. GitHub authentication failed to persist because of a permissions issue (/.config/gh) First, install the GitHub CLI if you don't have it:
   brew install gh
   Then, to authenticate run: gh auth login
   To verify it was correctly login check with:
   gh auth status

2. Under this error: mkdir `/Users/elicres/.config/gh: permission denied`
   The solution is:

   ```
   mkdir -p ~/.config/gh
   sudo mkdir -p ~/.config/gh
   sudo chown -R $USER ~/.config
   ```

3 Databases

Popular open data repositories:

- UC Irvine Machine Learning Repository
- Kaggle datasets
- Amazon's AWS datasets

Meta portals (they list open data repositories):

- http://dataportals.org/
- http://opendatamonitor.eu/
- http://quandl.com/

Other pages listing many popular open data repositories:

- Wikipedia's list of Machine Learning datasets
- Quora.com question
- Datasets subreddit

4 Exercises with Flask

4.1 Hello World

In Python use the following code:

```
from flask import Flask
app = Flask(__name__)
@app.route("/")
def home():
    return "Hello World!"
app.run(debug=True)
```

Run in the terminal with `python app.py`
Go to the address: `http://127.0.0.1:5000/` that is the local address as http is the protocol. 127.0.0.1 is own computer IP. 5000 is the port where Flask is listening.

4.2 HTML Templates

Create the structure Directory project app.py The file's name must be templates/ inside the file index.html

```
from flask import Flask, render_template
app = Flask(__name__)
@app.route("/")
def home():
    return render_template("index.html")
app.run(debug=True)
```

The directory Templates must be there! Then by default render will look for the specified file, index.html in this case. inside index.html: `<h1>My First Page?</h1>`

4.3 Dynamic Data

The main syntaxis in HTML used: double left-right key-brackets to show values and one key-bracket wuith percentage sign is to execute code. In the file index.html the next code is executed:

In the next code, the default request is GET, so it calls a render template using books.html to display the form. But after the form has been filled and que browser request POST, then the conditional if is activated, it calls the sql affects in the database and the return value is the function home.

```
@app.route('/books', methods=['GET', 'POST'])
def books():
    title = None
```

```
    author = None
    if request.method == 'POST':
        title = request.form['title']
        author = request.form['author']
        insert_book(title, author)
        return redirect(url_for('home'))
    return render_template("books.html",title=title,author=author)
```

4.4 Forms

Now users send data to Flask First in the file index.html write:

This HTML shows that the format <a always uses GET, which is dangerous for deleting. SO the correct manner for deleting is by using,
`<form method="post">` because it allows POST only, then it is in accordance with the code in python.

```
<ul>
    {% for boo in books %}
        <p> {{ boo[1] }} by {{boo[2]}}
            <a href="{{ url_for('edit_book', id=boo[0]) }}">
                Edit
            </a>

            <form action="{{ url_for('delete_book', id=boo[0]) }}" method="post" styl
                <button
                    type="submit"
                    #
                    onclick = "return confirm('Delete the book {{boo[1]}} by {{boo[2]
                    Delete book!
                </button>
            </form>
        </p>
    {% endfor %}
</ul>
```

For the python file:

```
from flask import Flask, render_template, request
```

```
app = Flask(__name__)
@app.route("/", methods=["GET", "POST"])
def home():
    if request.method == "POST":
        title = request.form["title"]
        return f"You entered: {title}"
    return """"""
app.run(debug=True)
```

4.5 Stylish

The page looks not pretty, but still can be fixed using templates. To do it I can use a file named style.css with is placed in the directory static. This contents the style for the HTML to be formatted and colored. It is link on the top of each HTML file with some very small snippet.

5 Webpage with Flask

Goal: Build a working web application that stores and manages data locally.

- Install once: Python 3.10+, Visual Studio Code Git Browser (Chrome / Firefox)

  You will use: Flask

  SQLite browser (to inspect database visually) Terminal (macOS Terminal / Windows PowerShell)

- Project Setup Create folder mkdir book-manager cd book-manager

- Initialize Git

  ‘‘‘bash
  git init

  Create .gitignore:

  touch .gitignore
  nano .gitignore

```text
__pycache__/
venv/
*.pyc
```

- Virtual environment
- Install Flask `pip install flask`
- Freeze dependencies:

To send all this into GitHub i activate the git locally, then git add .
git -m "arreglado"
git remote add origin `httpsURL`
This github url is copied from the same site. That means the files are now in GitHub.

6 Dockers

The creation of Duckers allows the app that is running in my laptop to be running properly in other servers and also in cloud servers. Like the other strategies, first I will install it locally The username used is eldockerqueuso and password is GuaG****?! All these is done in the Docker Desktop which is an app running in my laptop. The icon is a whale on. a blue background.

6.1 Local Ducker

- Write my Python application. Done
- Create a Dockerfile. Done this is the exact name *Dockerfile*
- Install Docker Desktop on my Mac. Done
- Build the image locally

The file name is Dockerfile Then the content is the one in the program. To actually running Docker first it needs to be installed. To verify that it is not there run `which docker`
Also `docker --version`
To install Docker in the terminal write the next:

`brew install --cask docker`
Run it:
`open -a Docker`
Docker needs a way to expose the web server port: EXPOSE 5000, and Flask should be listen on al network interfaces app.run(host="0,0,0,0", port =5000)
instead of: app.run()

using 0.0.0. tells Flask to accept connections from outside the container.

The line CMD ["python", "p1.py"] inside the Dockerfile means the command it is running.

After installed it verify with
`docker ps`

`docker build -t library-app .` No olvides el punto final!
docker run -p 5000:5000 library-app

For the next sessions. First I need to check the Docker is running, do it in the terminal with docker ps -a Then a list came like this:

```
CONTAINER ID   IMAGE         COMMAND          CREATED        STATUS                     PORTS     NAMES
2132c97a590c   library-app   "python p1.py"   40 hours ago   Exited (0) 39 hours ago            relaxed_banach
badba3616c34   library-app   "python p1.py"   40 hours ago   Exited (0) 40 hours ago            interesting_jemison
```

The names are invented by Docker. Now I can open some of them. For the next time I need to open write this:

docker start relaxed_banach
then go to the port http://localhost:5000

To stop it write: docker stop relaxed_banach
docker images

```
 IMAGE              ID            DISK USAGE   CONTENT SIZE   EXTRA
library-app:latest   69bd11828951       1.67GB          415MB    U
```

U means in use.

The overall process for verifying docker is running check the next list:

- I have a docker image (library-app)
- I have a running container (relaxed_banach)

- The app is currently available on http://localhost:5000
- All the changes are updated in Git.

Unlike Postgre or MySQL, SQLite does not run as a separate server, it is only a file.
To pen the container interactive shell use: docker exec -it relaxed_banach sh
This changes the prompt to a #

Explanation: docker exec runs a command inside the running container.
-i keep STDIN open.
-t allocate a pseudo-terminal
relaxedbanach is the container name
sh start a shell

Note: bash and sh are shells, i. e. programs that let me type commands and interact with a a Linux-Unix system. Once inside the container I can inspect the container with commands as:

- ls
- env #environmental variables
- ps #running process
- whoami #current user

6.2 Docker volume

The database will be build outside the container to avoid it fades whit the container image.
The image running inside the container is not persistent, that means it only works on the database inside the image. I need to add an instance to allow the database live inside the laptop. A **bind mount** is a way to make a file or directory from my host machine appear inside the docker container. Instead of copying files into the container, Docker gives the container direct access to a location on my computer.

Docker image and Docker container. Images and containers

- Create a file called .dockerignore to specify that the next time docker is run the database will not be charged into the docker.
- Stop and remove the current docker image with docker stop relaxed_banach then docker rm relaxed_banach

- With this commands every thing is initiallized again, this time considering the database book.db outside the docker.

```
docker run \
  --name library-app-container \
  -p 5000:5000 \
  -v "$(pwd)/books.db:/app/books.db" \
  library-app
```

- Start running again the new docker with docker start library-app-container and verify that the database book.db is there because it is in the laptop.

Apparently I hav learn these so far:

- Build images with docker build
- Run containers with docker run
- Start/Stop containers
- Understand image vs container
- Use bind mounts for SQLite persistence
- Use .dockerignore to control the build context
- Store code in GitHub

Note: docker run creates a new container and starts it. docker run –name myapp nginx the docker creates a new container from the nginx image, Applies settings -v -p environment variables, etc. and starts the container. That is equivalent to docker create –name myapp nginx then docker start myapp docker run IMAGE creates new container and start it
docker start CONTAINER start an existing container
docker create IMAGE create but does not start
docker stop CONTAINER stop a running CONTAINER.
compose.yaml
services: app: image: library-app container_name: library-app-container ports: - ”5000:5000” volumes: `- ./books.db:/app/books.db`
services: app: image: library-app container_name: library-app-container ports: - ”5000:5000” volumes: - .books.db:appbooks.db

6.3 yaml

Now replace docker run by YAML. The file is named compose.yaml the code inside it is

Lo que se corre es el contenedor por su nombre. The application can be run using: docker compose up

Once this is working. I can exit with CNTRL-D and then run it again this time with docker compose up -d

Check the status with docker compose ps

```
NAME                    IMAGE         COMMAND          SERVICE   CREATED          STATUS              PORTS
%library-app-container   library-app   "python p1.py"   app       34 minutes ago   Up About a minute   0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
```

Docker Compose is a tool for defining and running multi-container Docker applications using a YAML configuration file.

Instead of typing long docker run commands, you describe your application in a file (usually compose.yaml or docker-compose.yml) and start everything with one command.

I have a Dockerfile in my folder then this is a dynamic web application.

6.4 Next time I open the app

I must chose between using the VENV or using the ducker. For the case of ducker this the sequence:

- Open the Docker Desktop
- docker compose up -d
- docker compose ps

To close it use docker compose down

to run it locally I use: docker compose up

Notes:

The file compose.yaml. mounts books.db as a volume.

Flask-s built-in server is single-threaded and not meant for production. Render and most platforms expect a proper WSGI server. So, for production change the last line in Dockerfile from CMD [”python”, ”p1.py”]

to CMD [”gunicorn”, ”–bind”, 0.0.0.0:5000”, ”p1:app”]

p1:app means: file p1.py, Flask named app. and add gunicorn in the requirements.txt

6.5 Gunicorn and Render

Render is a platform where I am displaying the page with database, I go to the web render.com and enter using my GitHub account. I abandon Render before starting because for database persistency it costs some money. I will pay after I am ready with my initial learning. what is gunicorn?

6.6 Railway

https://bookmanager-production-db35.up.railway.app/